XIAMEN UNIVERSITY MALAYSIA
厦門大學馬來西亞分校

EEE211 Week4

Lab Practice

C Program to Compute the Eigenvalue and Eigenvector

Write a C program to implement the power method to find the most dominant eigenvalue λ_A and its corresponding eigenvector v_A of an N×N matrix using the power method. Display the result of each iteration until the algorithm converges.

// C Source code

```c
// powermethod.c
// Find the largest eigenvalue and its eigenvector using power method
#include <stdio.h>
#include <math.h>

#define N 3

void powerMethod(double A[][N], double v_old[], double v_new[]);
void multiplyMatrix(double A[][N], double v[], double result[]);
```

Function multiplyMatrix()

```c
void multiplyMatrix(double A[][N], double v[], double result[])
{
    for(int i=0; i<N; i++)
    {
        result[i] = 0.0;
        for(int j=0; j<N; j++)
        {
            result[i] += A[i][j]*v[j];
        }
    }
}
```

result[0] = A[0][0]*v[0] + A[0][1]*v[1] + A[0][2]*v[2];
result[1] = A[1][0]*v[0] + A[1][1]*v[1] + A[1][2]*v[2];
result[2] = A[2][0]*v[0] + A[2][1]*v[1] + A[2][2]*v[2];

Function powerMethod ()

```
22  void powerMethod(double A[][N], double v_old[], double v_new[])
23  {
24      double epsilon = 0.005;
25      double error = 10.0;
26      double new_eigenvalue = 0.0;
27      double old_eigenvalue = 0.0;
28      double largest = 0.0;
29      int itr = 0;
30
```

`epsilon`= ε
`error`= $|\lambda_{k-1} - \lambda_k|$
`new_eigenvalue`= λ_{k+1}
`old_eigenvalue`= λ_k
`largest`= maximum of the element in `v_new`
`itr`= iteration number

```
31      printf("\nIteration\t\t\tv(k)\t\t\t\teigenvalue(k)\t|eigenvalue(k+1)-eigenvalue(k)|\n");
32      while(error > epsilon)
33      {
34          old_eigenvalue = new_eigenvalue; // store the previous eigenvalue in old_eigenvalue
35          largest = 0.0;
36          printf("\t%d\t",itr); // print the current iteration number
37          for(int i=0; i<N; i++)
38          {
39              printf("%f\t", v_old[i]); // print the current eigenvector, v_old
40          }
```

Line 32: Keep running this loop if error > epsilon.
Line 34: Store the previous eigenvalue as the current eigenvalue.
Line 37-40: Print the current eigenvector elements

```
41          multiplyMatrix(A, v_old, v_new); // calculate new eigenvector v_new = A*v_old
42          for(int i=0; i<N; i++)
43          {
44              if(fabs(v_new[i]) > largest) // find the largest element in eigenvector v_new
45                  largest = v_new[i]; // store the largest eigenvector element value
46          }
47          new_eigenvalue = largest; // assign the largest element as the new eigenvalue
```

Line 41: Calculate `v_new= A * v_old`.
Line 42-46: Find the largest magnitude out of `v_new[0]`, `v_new[1]` and `v_new[2]`.
Line 47: Store the largest magnitude as the new eigenvalue λ_{k+1} in `new_eigenvalue`.
Note: The function `fabs()` is to find the **absolute** or the **magnitude** of a variable. This function is from the `math.h` library.

```
48          for(int i=0; i<N; i++)
49          {
50              v_old[i] = v_new[i]/new_eigenvalue; // divide every element of v_new with new_eigenvalue
51          }
52
53          printf("%f",new_eigenvalue); // print the new eigenvalue
54          error = fabs(new_eigenvalue-old_eigenvalue);// calculate the absolute error
55          if(itr != 0) printf("\t%f", error); // print the error value
56          printf("\n");
57          itr++;
58      }
```

Line 48-51: Calculate the new eigenvector using the equation:

$$v_{k+1} = \frac{1}{\lambda_{k+1}} A v_k$$

Line 54: Calculate the `error` $= |\lambda_{k+1} - \lambda_k|$

Line 57: Increment the iteration number, `itr` for the next iteration.

```
59      printf("\t%d\t", itr); // print the current iteration number
60      for(int i=0; i<N; i++)
61      {
62          printf("%f\t",v_old[i]); // print the current eigenvector
63      }
64
65      printf("\n\nThe largest eigenvalue is %f.\n", new_eigenvalue);// print the final eigenvalue
66      printf("The eigenvector is [");
67      for(int i=0; i<N; i++)
68      {
69          printf("%f ", v_old[i]); // print the final result of the eigenvector
70      }
71      printf("].\n");
72  }
```

This code is outside of the `while` loop after the `error < epsilon`.
Line 59-63: Print the eigenvector of the last iteration.
Line 65: Print the final result of the largest eigenvalue.
Line 67-70: Print the final result of the eigenvector.

Output

```
Iteration               v(k)                            eigenvalue(k)   |eigenvalue(k+1)-eigenvalue(k)|
        0       0.000000        1.000000        0.000000        3.000000
        1       -0.666667       1.000000        -0.333333       4.000000        1.000000
        2       -0.916667       1.000000        -0.416667       4.333333        0.333333
        3       -0.980769       1.000000        -0.423077       4.403846        0.070513
        4       -0.995633       1.000000        -0.419214       4.414847        0.011001
        5       -0.999011       1.000000        -0.416419       4.415430        0.000583
        6       -0.999776       1.000000        -0.415099

The largest eigenvalue is 4.415430.
The eigenvector is [-0.999776 1.000000 -0.415099 ].

Process returned 0 (0x0)   execution time : 2.735 s
Press any key to continue.
```